

Lab4 实验报告

一、思考题

Thinking 4.1

1. 内核在保存现场的时候是如何避免破坏通用寄存器的？

- 使用 `k0 / k1` 这两个**内核专用寄存器**（用户态不会用，异常时也不会保存）
- 先用 `k0 / k1` 暂存 `CP0` 寄存器的值，再把 `k0 / k1` 存到栈上
- 这样就不会破坏任何通用寄存器（`$0-$31`）的值

代码： `SAVE_ALL` 宏中 `mfc0 k0, CP0_STATUS` → `sw k0, TF_STATUS(sp)`

关于 TrapFrame

2. 系统陷入内核调用后可以直接从当时的 `$a0-$a3` 参数寄存器中得到用户调用 `msyscall` 留下的信息吗？

不能！

- 陷入内核后，`a0—a3` 的值已经被保存到 `Trapframe` 里了
- 内核需要从 `tf->regs[4] ~ tf->regs[7]` 读取，而不是直接读寄存器

3. 我们是怎么做到让 `sys` 开头的函数"认为"我们提供了和用户调用 `msyscall` 时同样的参数的？

- `do_syscall` 从 `Trapframe` 把 `a0`（syscall号）、`a1-a3`（参数）提取出来
 - 再从用户栈 `tf->regs[29] + 16/+20` 取出第4、5个参数
 - 把这些参数按顺序传给 `sys_xxx` 函数，就好像用户直接调用一样
-

4. 内核处理系统调用的过程对 `Trapframe` 做了哪些更改？这种修改对应的用户态的变化是什么？

- `tf->cp0_epc += 4`：跳过 `syscall` 指令本身，避免死循环
 - `tf->regs[2] = 返回值`：把系统调用返回值存入 `$v0`
 - 用户态 `eret` 返回后，PC 就是 `syscall` 下一条指令，`$v0` 就是返回值
-

Thinking 4.2

为什么 `envid2env` 中需要判断 `e->env_id != env_id` 的情况？如果没有这步判断会发生什么情况？

- **原因：** 同一个数组槽位可能被复用。进程销毁后槽位被回收，新进程可能分配到同一个槽位（同一个 `e` 地址），但 `env_id` 是不同的。
 - **没有这步会：** 拿着旧进程的 `env_id` 去操作新进程，造成权限越界。
-

Thinking 4.3

`mkenvid()` 函数不会返回 0，请结合系统调用和 IPC 部分的实现与 `envid2env()` 函数的行为进行解释。

- `envid2env` 中约定：**`env_id = 0` 表示当前进程**（`curenv`）
 - 如果 `mkenvid()` 可能返回 0，就无法区分"当前进程"和" id=0 的那个进程"了
 - 所以 `mkenvid` 从 1 开始递增，保证 0 永远不会被分配，保留为"指代自身"的特殊值
-

Thinking 4.4

关于 `fork` 函数的两个返回值，下列说法正确的是？

- `sys_exofork` 只在父进程执行一次
 - 内核创建子进程，复制 `Trapframe`，把父进程 `EPC` 和 `$v0` 都复制过去
 - 两个进程分别从 `eret` 返回：
 - 父进程：返回子进程 `pid`
 - 子进程：返回 0（因为内核手动设了 `e->env_tf.regs[2] = 0`）
 - 所以是一次调用，两个返回
-

Thinking 4.5

1. 为什么子进程的 `UXSTACK` 必须重新分配，而不能共享或设为 `COW`？

- `UXSTACK` 是**每个进程私有的异常栈**，用来处理 `TLB Mod` 等异常
 - 如果共享，父子进程同时发生异常时会互相覆盖栈内容
 - 如果设为 `COW`，异常发生时栈访问触发 `TLB Mod`，又要往栈上写，**死循环**
-

2. 为什么不对 UVPT/VPT/VPD 等自映射区域调用 duppage? 如果误处理, 可能引发哪些不一致或错误行为?

- UVPT/VPT/VPD 是**自映射区域**, 由硬件根据页目录基址自动映射, 不是实际的物理页
- `duppage` 对普通物理页做 COW, 但自映射区域"映射的是页表本身", 父子进程有独立的页目录
- 误处理会导致: 子进程看到的页表不是自己的, 修改 vpt 会意外修改父进程的页表, 内存隔离失效

Thinking 4.6

1. vpt 和 vpd 的作用是什么? 怎样使用它们?

- `vpt` (Virtual Page Table) : 把整个页表当一个数组来访问, `vpt[VPN]` 就是对应虚拟页的 PTE
- `vpd` (Virtual Page Directory) : 访问页目录, `vpd[PDX(va)]` 就是页目录项
- 使用: 直接当指针解引用, 比如 `pte_t pte = vpt[va >> PGSHIFT]`

2. 从实现的角度谈一下为什么进程能够通过这种方式来存取自身的页表?

- 页目录的**最后一项指向页目录本身** (自映射)
- 这样 `UVPT + 4*VPN` 这个虚拟地址, 经过 MMU 两级翻译后, 正好指向页表中对应的 PTE
- 硬件帮我们完成了地址翻译, 软件就可以像访问普通内存一样访问页表

3. 它们是如何体现自映射设计的? 进程能够通过这种方式来修改自己的页表项吗?

- 体现: 页目录最后一项 $PDX(UVPT) = \text{页目录的物理地址} \mid \text{权限位}$, 形成"自己指向自己"的环
- **用户态不能修改**: 因为这些页表页的 PTE 没有 `PTE_D` (写权限), 用户态写会触发 TLB Mod
- 只能通过 `sys_mem_map` 等系统调用让内核修改

Thinking 4.7

1. 什么时候会出现"异常重入"?

- 当异常处理程序本身又触发异常时。比如:
 1. 用户态写 COW 页, 触发 TLB Mod 异常
 2. 内核把 Trapframe 复制到 UXSTACK 上返回用户态处理
 3. 用户态异常处理函数执行时, 如果触发了另一个异常 (比如缺页)
- 这就需要第二个异常的现场不覆盖第一个异常的现场

2. 内核为什么需要将异常的现场 Trapframe 复制到用户空间？

- 异常处理逻辑是**用户态实现**的（`cow_entry`），不是内核态
- 用户态处理函数需要知道：哪个地址触发了异常、当时各寄存器的值、EPC 是多少
- 复制到 UXSTACK 上，用户态函数就能直接用指针访问 Trapframe 了

Thinking 4.8

在用户态处理页写入异常，相比于在内核态处理有什么优势？

1. **微内核设计思想**：把策略（什么时候复制、怎么复制）放到用户态，内核只提供机制
2. **减少内核代码复杂度**：内核不用处理各种复杂的页错误情况
3. **更灵活**：不同进程可以注册不同的异常处理函数
4. **安全性**：异常处理在用户态运行，崩溃了只会影响当前进程，不会整个系统挂掉

Thinking 4.9

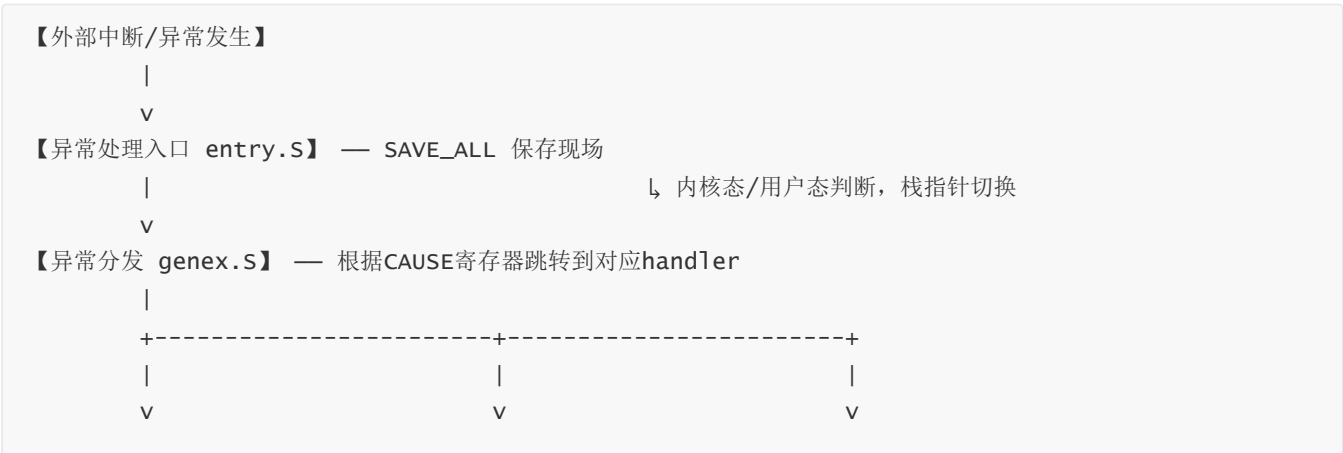
为什么在设置写时复制保护机制之前，需通过 `syscall_set_tlb_mod_entry` 设置父进程页写入异常处理函数？若在写时复制保护机制完成之后再设置会怎样？

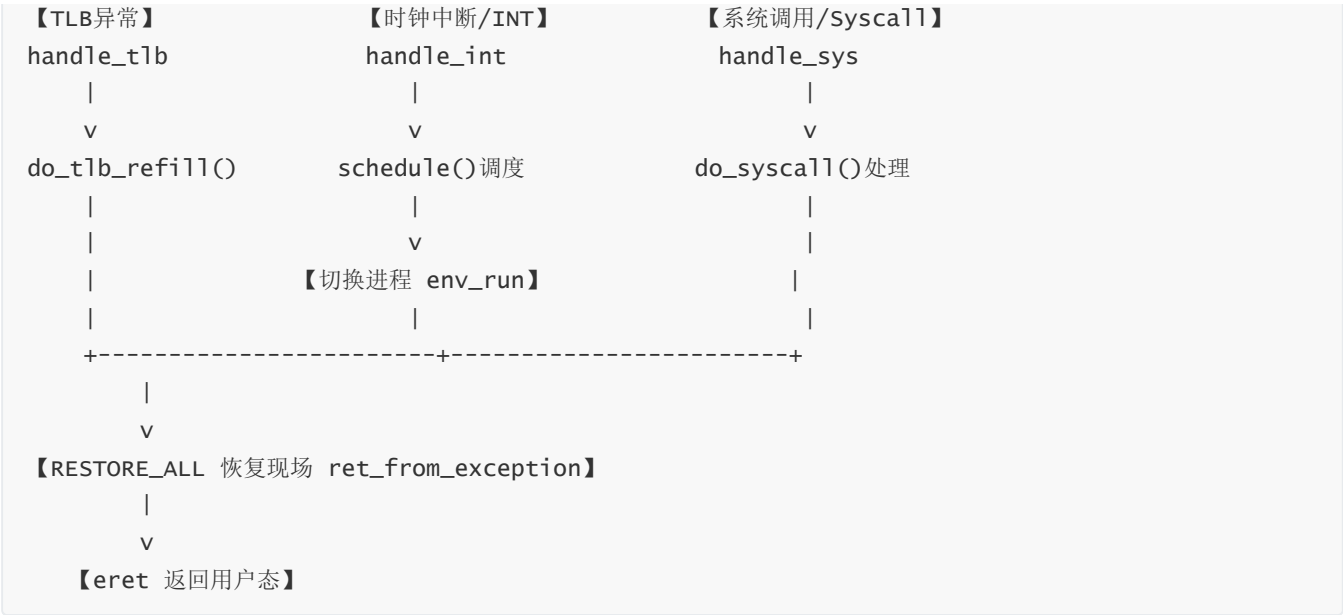
- **原因**：设置 COW 保护后，任何写操作都会立即触发 TLB Mod 异常，如果先设置 COW 再注册处理函数，中间有一个**窗口**，此时异常触发了但没有处理函数，直接 panic
- **之后设置会**：出现竞态条件！`duppage` 取消了 `PTE_D` → 还没来得及 `sys_set_tlb_mod_entry` → 父进程写了那个页 → 触发 TLB Mod → 处理函数为空 → `panic("TLB Mod but no user handler registered")`

二、难点分析与实验体会

中断/异常处理核心流程

整体执行路径总览：





- 关键对比：Trapframe 位置约定：
- 所有异常发生时，现场都保存在 **KSTACKTOP** 下方固定位置
 - 用户态进入异常：sp 从用户栈切换到内核栈 KSTACKTOP
 - 内核态进入异常：直接在当前内核栈上继续生长

1. 核心难点分析与解决思路

难点一：Syscall 参数传递与返回值机制

问题描述：
系统调用需要从用户态传递最多5个参数到内核态，还要返回结果。但进入内核后，a0-a3寄存器的值已经被保存到Trapframe中，不能直接使用。

解决思路：

```
void do_syscall(struct Trapframe *tf) {
    int sysno = tf->regs[4];           // a0 = syscall number
    u_int arg1 = tf->regs[5];          // a1 = 第1个参数
    u_int arg2 = tf->regs[6];          // a2 = 第2个参数
    u_int arg3 = tf->regs[7];          // a3 = 第3个参数
    // 第4、5个参数从用户栈读取
    u_int arg4 = *((u_int *)tf->regs[29] + 4);
    u_int arg5 = *((u_int *)tf->regs[29] + 5);

    // 查表调用，返回值存入 $v0
    tf->regs[2] = func(arg1, arg2, arg3, arg4, arg5);
}
```

- 核心坑点：tf->cp0_epc += 4
- syscall触发时，EPC指向syscall指令本身
 - 如果不加4，eret后又会执行syscall，造成死循环！

难点二：进程切换时的栈指针魔法

问题描述：

时钟中断触发调度时，如何实现从旧进程到新进程的无缝切换。

解决思路：

```
void env_run(struct Env *e) {
    // Step 1: 保存旧进程现场 (Trapframe就在KSTACKTOP-1位置
    if (curenv) {
        curenv->env_tf = *((struct Trapframe *)KSTACKTOP - 1);
    }
    // Step 2: 切换当前进程
    curenv = e;
    // Step 3: 切换页表
    cur_pgdir = e->env_pgdir;
    // Step 4: 加载新进程现场
    env_pop_tf(&curenv->env_tf, curenv->env_asid);
}
```

```
LEAF(env_pop_tf)
    mtc0    a1, CP0_ENTRYHI    // 切换ASID
    move    sp, a0              // 关键! sp指向新进程的env_tf
    j       ret_from_exception  // RESTORE_ALL自然恢复新进程
END(env_pop_tf)
```

关键理解：进程切换的本质是

- **保存**：把KSTACKTOP处的Trapframe拷贝到旧进程的Env结构体
- **恢复**：把新进程Env的env_tf地址赋给sp，RESTORE_ALL自动恢复

难点三：fork 的"一次调用，两次返回"

问题描述：

sys_exofork 只执行一次，但父子进程分别返回不同的值。

解决思路：

```
int sys_exofork(void) {
    struct Env *e;
    env_alloc(&e, curenv->env_id);

    // 从内核栈上的当前Trapframe完整拷贝给子进程
    e->env_tf = *((struct Trapframe *)KSTACKTOP - 1);

    // 子进程返回值 = 0! (通过修改子进程的Trapframe实现)
    e->env_tf.regs[2] = 0;

    e->env_status = ENV_NOT_RUNNABLE;
    return e->env_id; // 父进程返回子进程ID
}
```

理解关键：两个进程的区别只在于**Trapframe的regs[2]字段不同。

难点四：IPC 接收方唤醒机制

问题描述：

如何实现"接收方阻塞等待，发送方唤醒接收方"的同步机制。

解决思路：

```
【接收方】sys_ipc_recv(dstva)
|
v
env_ipc_recving = 1    // 标记接收状态
env_status = NOT_RUNNABLE
从调度队列移除
|
v
schedule() → 切换到其他进程，阻塞

                【发送方】sys_ipc_try_send(whom, val, ...)
                |
                v
                检查对方env_ipc_recving == 1?
                |
                v
                填写对方的env_ipc_value/from/perm
                将对方env_status = RUNNABLE
                把对方重新加入调度队列
```

2. 实验收获与技巧总结

2.1 C 语言技巧：系统调用表的设计

```
// 利用C语言数组初始化语法，实现O(1)跳转
void *syscall_table[MAX_SYSNO] = {
    [SYS_putchar] = sys_putchar,
    [SYS_print_cons] = sys_print_cons,
    [SYS_getenv] = sys_getenv,
    [SYS_yield] = sys_yield,
    // ... 共15个系统调用
};
```

设计思想：用syscall number作为数组索引，直接跳转到对应处理函数。

2.2 权限检查 checkperm 的规律

系统调用类型	checkperm 值	说明
内存操作 (mem_alloc/map/unmap)	1	只能操作自己或子进程
进程管理 (env_destroy)	1	只能销毁自己或子进程
IPC 发送 (ipc_try_send)	0	可以发给任意进程! 不限于父子

- 除了IPC相关的syscall，都需要设为1。

2.3 微内核设计思想的体现

- **内核只提供机制，策略在用户态实现
 - 例如：IPC 内核只实现try_send，重试循环在用户态
 - COW异常处理在用户态的 cow_entry
- **内核数据结构暴露给用户态
 - PCB (Env结构体) 直接映射到用户态的 envs[] 数组
 - 页表通过自映射区域 vpt[]、vpd[] 暴露

2.4 异常不可重入的设计

为什么异常不可重入的三个保障机制：

- 进入内核立即 关中断 (清除STATUS_IE位)
- Trapframe只有一份在KSTACK栈顶
- 大量依赖全局状态 (curenv、cur_pgdire、调度队列)

设计选择：简化内核实现，不支持内核抢占。

2.5 用户程序的执行入口链路

```
用户写的 main()
  ↑ 被调用
libmain() 在 libos.c
  ↑ 被调用
_start() 在 entry.S (汇编入口, 在Linker script中定义)
  ↓
用户栈初始化:
    lw      a0, 0(sp)    # argc
    lw      a1, 4(sp)    # argv
    jal     libmain
```

```
void libmain(int argc, char **argv) {
    // 初始化全局env指针
    env = &envs[ENVX(syscall_getenvid())];
    // 调用用户主函数
    main(argc, argv);
    // 优雅退出
    exit();
}
```

3. 实验注意事项与避坑指南

要点	现象	解决方案
忘记 EPC 没加4	syscall 死循环	do_syscall 第一行必须 <code>tf->cp0_epc += 4</code>
TLB Mod 处理函数为空	写COW页触发异常后panic	fork前必须先 <code>sys_set_tlb_mod_entry</code>
envid2env 权限错误	IPC发送失败	IPC的checkperm必须传0
子进程 UXSTACK 没分配	子进程写COW页时缺页	fork时单独给子进程分配UXSTACK
自映射区域被duppage	子进程看到父进程的页表	duppage时跳过UVPT以上区域
Trapframe位置记错	进程切换后寄存器混乱	记住: <code>(struct Trapframe *)KSTACKTOP - 1</code>

三、原创说明

本报告基于实验指导书和代码实现完成, 以及助教讲解视频